E X E R C I S E 0 3 · S Q L F O U N D A T I O N S SQL CASE Statements

Question 01 TABLE CREATION:

```sql
create CATALOG if not exists family;
create SCHEMA if not exists family.gauteng;
create table if not exists family.gauteng.products (product_id INT ,product_name STRING ,price
INT);
INSERT INTO family.gauteng.products VALUES (1,'Laptop',1200),(2,'Phone',800),(3,'Keyboard',45),(4, 'Monitor',300 ),(5, 'Mouse', 25);

SELECT * FROM family.gauteng.products;
```

Add parameter

Table +

	product_id	product_name	price
1	1	Laptop	1200
2	2	Phone	800
3	3	Keyboard	45
4	4	Monitor	300
5	5	Mouse	25

Question 01: **Classify each product by price into the three tiers below. 'Expensive' if price > 1000 'Mid-range' if price between 100 and 1000 'Budget' if price < 100 E X P E C T E D C O L U M N S : product_name price price_category**

```sql
--Question 01: Classify each product by price into the three tiers below,'Expensive' if price > 1000,'Mid-range' if price between 100 and 1000'Budget' if price < 100 --expected col:are product name,price,price_category

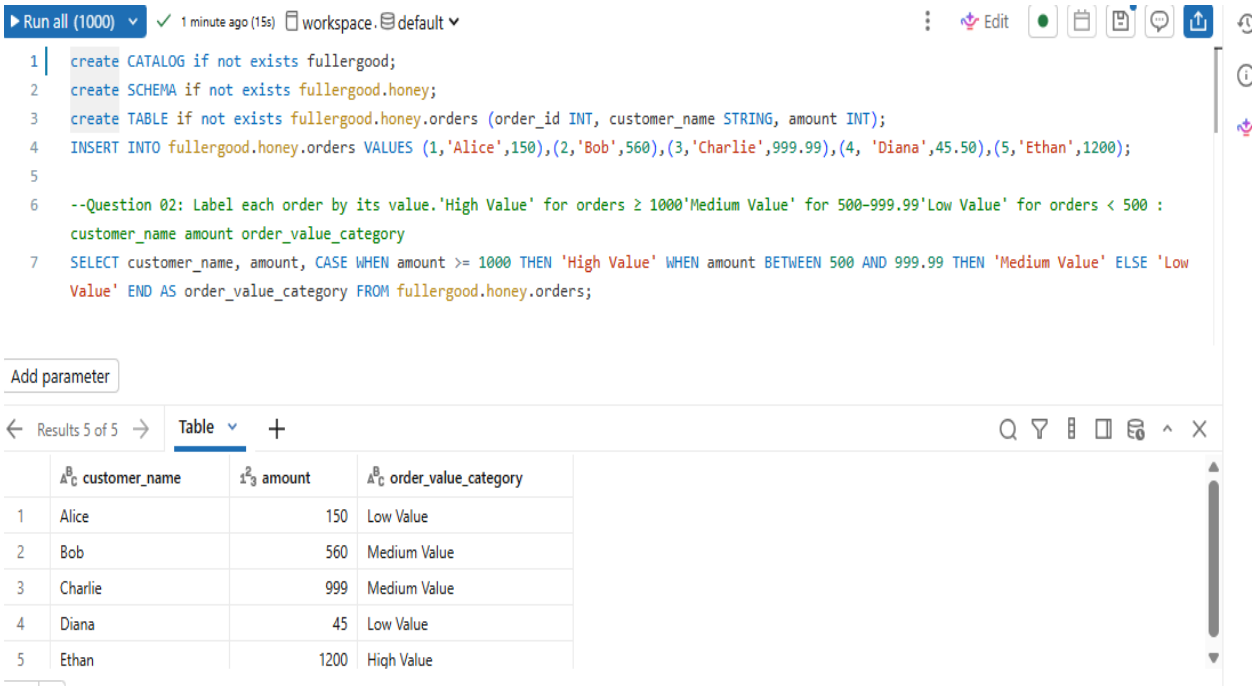
SELECT product_name,price,
CASE WHEN price > 1000 THEN 'Expensive'
WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
WHEN price < 100 THEN 'Budget'
END AS price_category
FROM family.gauteng.products;
```

Add parameter

Table +

	product_name	price	price_category
1	Laptop	1200	Expensive
2	Phone	800	Mid-range
3	Keyboard	45	Budget
4	Monitor	300	Mid-range
5	Mouse	25	Budget

Question 02: **Label each order by its value.'High Value' for orders ≥ 1000'Medium Value' for 500–999.99'Low Value' for orders < 500 : customer_name amount order_value_category**

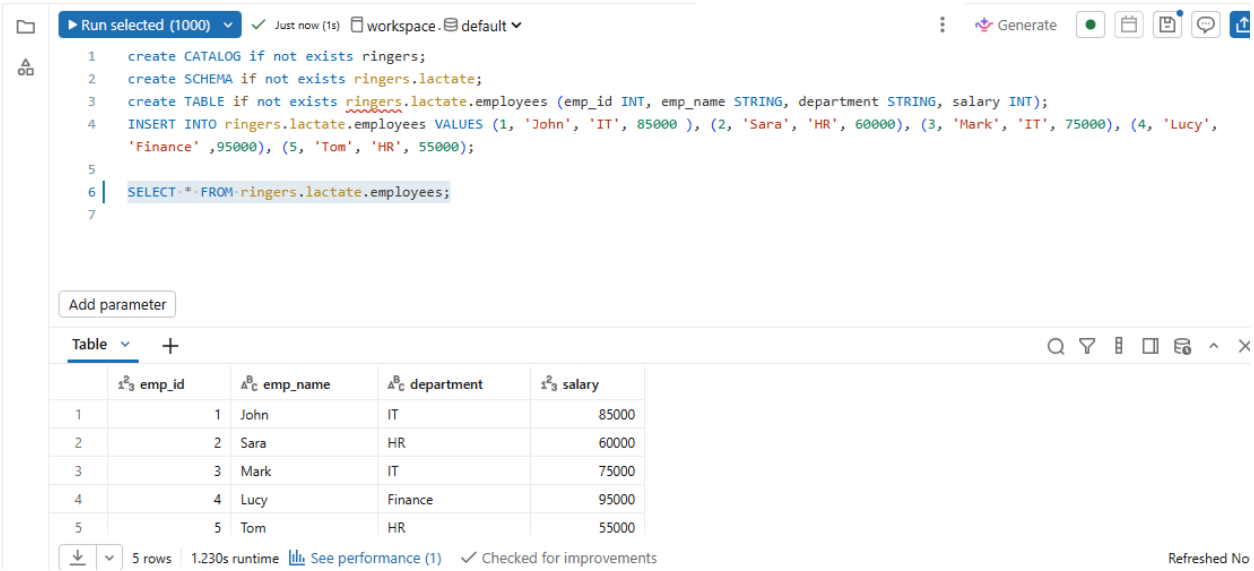


```
create CATALOG if not exists fullergood;
create SCHEMA if not exists fullergood.honey;
create TABLE if not exists fullergood.honey.orders (order_id INT, customer_name STRING, amount INT);
INSERT INTO fullergood.honey.orders VALUES (1,'Alice',150),(2,'Bob',560),(3,'Charlie',999.99),(4, 'Diana',45.50),(5,'Ethan',1200);

--Question 02: Label each order by its value.'High Value' for orders ≥ 1000'Medium Value' for 500-999.99'Low Value' for orders < 500 :
customer_name amount order_value_category
SELECT customer_name, amount, CASE WHEN amount >= 1000 THEN 'High Value' WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium Value' ELSE 'Low
Value' END AS order_value_category FROM fullergood.honey.orders;
```

	customer_name	amount	order_value_category
1	Alice	150	Low Value
2	Bob	560	Medium Value
3	Charlie	999	Medium Value
4	Diana	45	Low Value
5	Ethan	1200	High Value

Question 03: TABLE CREATION



```
create CATALOG if not exists ringers;
create SCHEMA if not exists ringers.lactate;
create TABLE if not exists ringers.lactate.employees (emp_id INT, emp_name STRING, department STRING, salary INT);
INSERT INTO ringers.lactate.employees VALUES (1, 'John', 'IT', 85000 ), (2, 'Sara', 'HR', 60000), (3, 'Mark', 'IT', 75000), (4, 'Lucy',
'Finance' ,95000), (5, 'Tom', 'HR', 55000);

SELECT * FROM ringers.lactate.employees;
```

	emp_id	emp_name	department	salary
1	1	John	IT	85000
2	2	Sara	HR	60000
3	3	Mark	IT	75000
4	4	Lucy	Finance	95000
5	5	Tom	HR	55000

Question 3: **Categorize each employee's position level using both department and salary. If in 'IT' and salary > 80000 → 'Senior IT' If in 'HR' and salary > 55000 → 'Experienced HR' Otherwise → 'Staff' E X P E C T E D C O L U M N S : emp_name department salary position_level**

```
→ 'Staff' :: emp_name department salary position_level

SELECT emp_name, department, salary,
CASE WHEN department = 'IT' AND salary > 80000 THEN 'Senior IT' WHEN department = 'HR' AND salary
> 55000 THEN 'Experienced HR'
ELSE 'Staff'
END AS position_level
FROM ringers.lactate.employees;
```

Add parameter

Results 6 of 6 | Table

	emp_name	department	salary	position_level
1	John	IT	85000	Senior IT
2	Sara	HR	60000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	Finance	95000	Staff
5	Tom	HR	55000	Staff

Question 04:TABLE CREATION

```sql
create CATALOG if not exists state;
create SCHEMA if not exists state.welkom;
create TABLE if not exists state.welkom.students ( student_id INT, student_name STRING, score INT);
INSERT INTO state.welkom.students VALUES (1,'Anna', 92),
(2, 'Ben', 76),
(3, 'Cara', 59),
(4, 'David', 83),
(5 ,'Ella',68);
SELECT * FROM state.welkom.students;
```

Add parameter

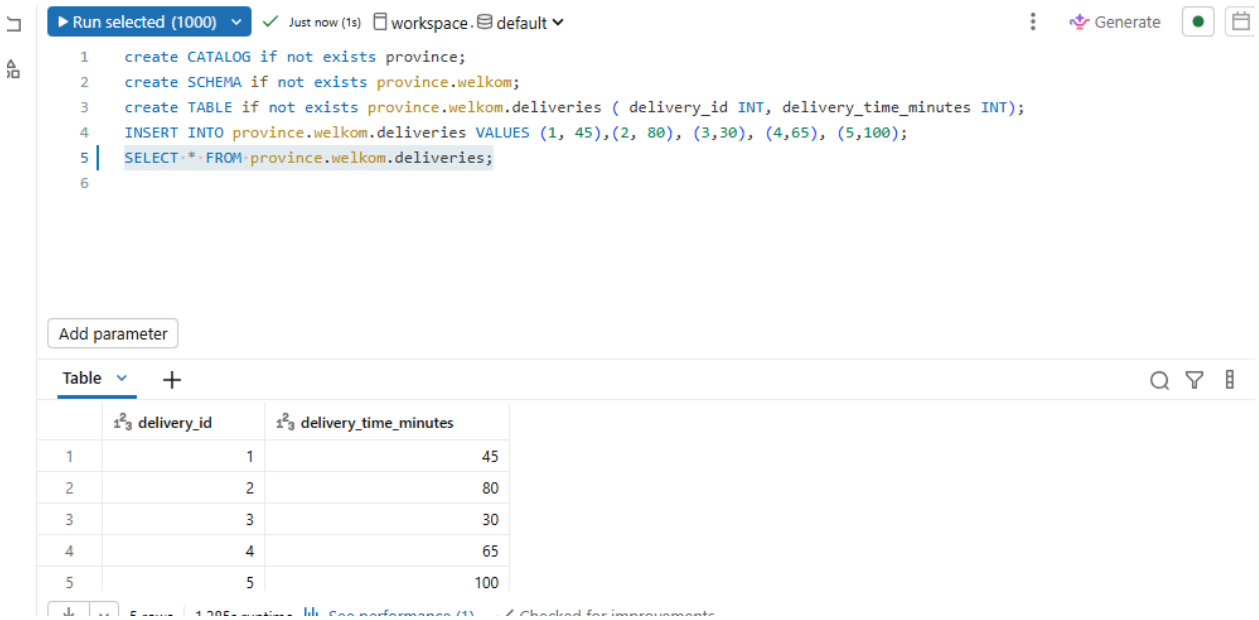
Table

	student_id	student_name	score
1	1	Anna	92
2	2	Ben	76
3	3	Cara	59
4	4	David	83
5	5	Ella	68

QUESTION 04:Assign each student a letter grade based on their score. ≥ 90 → 'A' 80–89 → 'B' 70–79 → 'C' 60–69 → 'D' < 60 → 'F' E X P E C T E D C O L U M N S : student_name score grade

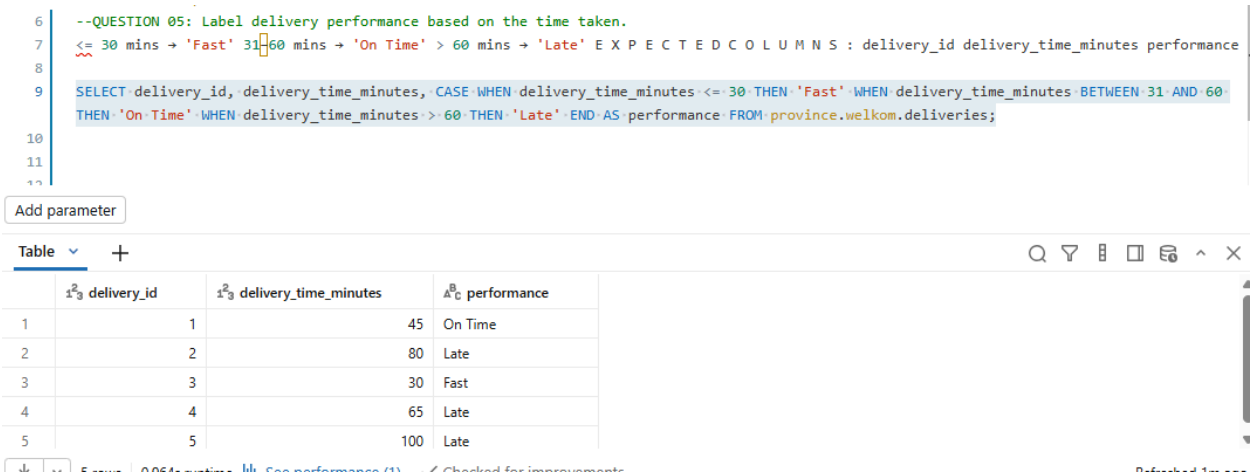


QUESTION 05: TABLE CREATION



	delivery_id	delivery_time_minutes
1	1	45
2	2	80
3	3	30
4	4	65
5	5	100

QUESTION 5:Label delivery performance based on the time taken. ≤ 30 mins → 'Fast' 31–60 mins → 'On Time' > 60 mins → 'Late' E X P E C T E D C O L U M N S : delivery_id delivery_time_minutes performance



	delivery_id	delivery_time_minutes	performance
1	1	45	On Time
2	2	80	Late
3	3	30	Fast
4	4	65	Late
5	5	100	Late